

LABORATORIO 03

¿Y quién dice que esa es la config?

Configuración efectiva de un broker.

Guía para hacerlo por tu cuenta, paso a paso

Confluent Platform 8.2.0 · Kafka 4.2 en modo KRaft · 3 brokers

Duración estimada · 45 minutos

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Al final de esta guía vas a haber **cambiado un parámetro de un servidor en caliente**, vas a haber comprobado que el cambio funciona, y vas a haber visto que **el archivo de configuración no se enteró de nada**.

Ese hueco entre lo que el archivo dice y lo que el servidor hace tiene nombre, y es la causa de una clase entera de incidentes.

LO QUE ESTE LABORATORIO DEMUESTRA

El archivo dice lo que alguien declaró. Solo el broker sabe lo que está usando.

CONCEPTO · DRIFT DE CONFIGURACIÓN

La **distancia entre lo que está escrito y lo que está corriendo**. Aparece cada vez que alguien cambia algo en caliente y no lo escribe en ningún sitio. **No hay ninguna alarma que avise de que existe**, y solo se descubre comparando las dos cosas a mano.

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Haber hecho los laboratorios 01 y 02	Hoy se da por sabido qué es un <code>docker-compose.yml</code> y qué es un quórum
Docker Desktop corriendo, con 6 GB de RAM	Ajustes de Docker Desktop, pestaña <i>Resources</i>
Git Bash abierto	<code>cd Capitulo_3/lab-03-configuracion-brokers</code>
Los puertos 9092, 9093 y 9094 libres	El paso 1 te avisa si no lo están

El caso

Un viernes por la tarde alguien de **NovaTech Logistics** sube un parámetro del broker en caliente para salir de un atasco. Funciona, el atasco se resuelve, y nadie lo escribe en ninguna parte.

Tres meses después el servidor se reinicia por un corte de luz. **El parámetro vuelve a su valor de antes**, porque el archivo nunca supo del cambio, y el atasco vuelve. Nadie relaciona una cosa con la otra, porque entre las dos hay tres meses.

El problema no fue el cambio en caliente, que estuvo bien hecho. **El problema es que nadie sabe leer las tres capas de la configuración de un broker**, y por eso nadie podía ver que el archivo y el servidor decían cosas distintas.

UN AVISO · DESDE AQUÍ CAMBIA LA METÁFORA DEL CURSO

Hasta el laboratorio 02 veníamos hablando de **camiones y flota**. Servía muy bien para explicar la identidad del clúster y el quórum.

Desde este laboratorio Kafka es un restaurante, porque lo que viene se explica mejor con mozos y comandas que con camiones. **No te has equivocado de material**. Es el mismo curso, y la metáfora se queda en el restaurante de aquí hasta el final.

LA METÁFORA · LAS TRES CAPAS, EN EL RESTAURANTE

El **mozo** que toma y entrega los pedidos es el **broker**, o sea el servidor. Un **tipo de comanda** es el **tópico**. Los **sectores del salón** son las **particiones**.

Y hoy nos interesa **cómo llega una instrucción a la cocina**, que pasa por tres formas distintas.

Está **la pizarra del menú del día**, que escribe el dueño. Está **la comanda impresa** que sale del sistema y llega a la cocina. Y está **lo que el cocinero tiene de verdad en la plancha ahora mismo**. Casi siempre las tres coinciden. **El día que no coinciden, hay que saber cuál manda**.

Las tres etapas, en Kafka

Etapa	Dónde vive	En el restaurante
1 • Lo que escribes	<code>docker-compose.yml</code> , como <code>KAFKA_NODE_ID: 1</code>	La pizarra del menú
2 • Lo que se tradujo	<code>/etc/kafka/kafka.properties</code> , como <code>node.id=1</code>	La comanda impresa
3 • Lo que el broker usa	En memoria, y se le pregunta al clúster	Lo que hay en la plancha

Las etapas 1 y 2 se leen en archivos. La etapa 3 se le pregunta al servidor. Y esa diferencia es todo el laboratorio.

LA REGLA DE TRADUCCIÓN DE LA ETAPA 1 A LA 2

En la imagen de Confluent no editas un `server.properties` a mano. Declaras variables `KAFKA_*` en tu compose y **la imagen las traduce al arrancar.**

La regla son tres pasos. **Quitar el prefijo `KAFKA_` , pasar a minúsculas, y cambiar cada `_` por un punto.**

Variable de entorno	Propiedad resultante
<code>KAFKA_NODE_ID</code>	<code>node.id</code>
<code>KAFKA_PROCESS_ROLES</code>	<code>process.roles</code>
<code>KAFKA_LOG_DIRS</code>	<code>log.dirs</code>
<code>KAFKA_MIN_INSYNC_REPLICAS</code>	<code>min.insync.replicas</code>

Levantar el clúster

Los comandos

```
cd Capitulo_3/lab-03-configuracion-brokers
chmod +x bin/*.sh kafka-cli/*.sh
bin/start-lab.sh
```

bin/start-lab.sh — levanta el clúster de tres brokers del laboratorio 02. **Hoy no vas a escribir ningún compose**, porque el trabajo es leer, no construir

Comprueba que están los tres

```
docker ps --filter name=kafka-broker
```

VAS BIEN SI...

Salen **tres líneas** con `Up`, y los puertos 9092, 9093 y 9094.

SI ALGO FALLA AL LEVANTAR

Es lo mismo del laboratorio 02. **Docker apagado**, ábrelo. `port is already allocated`, hay otro lab del curso arriba y hay que bajarlo. **Timeout de brokers**, sube Docker a 6 GB.

Etapa 2 · encontrar el archivo que manda

Qué vamos a hacer

La etapa 1 ya la conoces, porque son las variables `KAFKA_*` que escribiste en el laboratorio 02. Vamos directo a la etapa 2, **a buscar en qué archivo acabaron.**

Y ahí hay una sorpresa que a mucha gente le cuesta una tarde.

El comando

```
kafka-cli/list-config-files.sh
```

`kafka-cli/list-config-files.sh` — un envoltorio del curso. Lista los `.properties` del contenedor **y además le pregunta al proceso de Kafka con cuál de ellos arrancó de verdad.** Esa respuesta no se deduce, sale de la línea de comandos del `java` que está corriendo

LOS ARCHIVOS QUE HAY

```
/etc/kafka/broker.properties
/etc/kafka/connect-console-sink.properties
/etc/kafka/connect-console-source.properties
/etc/kafka/connect-distributed.properties
/etc/kafka/connect-file-sink.properties
/etc/kafka/connect-file-source.properties
/etc/kafka/connect-mirror-maker.properties
/etc/kafka/connect-standalone.properties
/etc/kafka/consumer.properties
/etc/kafka/controller.properties
/etc/kafka/kafka.properties
/etc/kafka/producer.properties
/etc/kafka/server.properties
```

Trece archivos. Y ahí, en la lista, está `server.properties`, que es el nombre que cualquiera diría si le preguntas cuál usa un broker de Kafka.

Ahora pregúntale al proceso

```
docker exec kafka-broker-1 sh -c "ps -o args= -C java | tr ' ' '\n' | grep properties"
```

`ps -o args= -C java` — la línea de comandos completa del proceso `java` que está corriendo
`tr ' ' '\n'` — parte esa línea en una palabra por renglón, porque es larguísima

grep properties — se queda con la única que nos interesa. **Esto no es una suposición, es lo que el proceso tiene en su línea de arranque**

LO QUE SALE

```
/etc/kafka/kafka.properties
```

AQUÍ ESTÁ LA SORPRESA

El broker no arrancó con `server.properties` . Arrancó con `kafka.properties` , que está en la lista pero no es el nombre que nadie esperaba.

Y ese archivo **no lo escribió nadie. Lo genera la imagen al arrancar**, traduciendo las variables `KAFKA_*` de tu compose. Si apagas el contenedor y miras la imagen, ahí solo hay **doce** archivos. El decimotercero nace con el arranque.

Compruébalo tú

```
docker exec kafka-broker-1 sh -c "ls /etc/kafka/*.properties | wc -l"
docker run --rm confluentinc/cp-kafka:8.2.0 sh -c "ls /etc/kafka/*.properties | wc -l"
```

docker exec — dentro del contenedor **que está corriendo**

docker run --rm — un contenedor nuevo de la misma imagen, que ejecuta el `ls` y muere. **Como no arranca Kafka, nadie genera el archivo**

LO QUE SALE

```
13    <- el contenedor corriendo
12    <- la misma imagen, sin arrancar Kafka
```

VAS BIEN SI...

Trece contra doce, y el proceso apunta a `kafka.properties` .

Retén esto, porque en el laboratorio o8b, sobre un servidor RHEL de verdad, vas a ver que ahí son **doce** y el archivo que manda **sí** es `server.properties` . **El decimotercero es una cosa de la imagen de Docker, no de Kafka.**

Y mira la traducción, con tus propias variables

```
docker exec kafka-broker-1 sh -c \  
    "grep -E '^(node.id|process.roles|log.dirs|min.insync.replicas)=' \  
    /etc/kafka/kafka.properties"
```

LO QUE SALE

```
log.dirs=/var/lib/kafka/data  
min.insync.replicas=2  
node.id=1  
process.roles=broker,controller
```

LAS HUELLAS QUE DELATAN AL ARCHIVO REAL

Fíjate en `log.dirs=/var/lib/kafka/data` y en el `node.id=1`. **Esos valores los pusiste tú en el compose.** La distribución de Kafka no puede saber dónde quieres los datos ni cómo se llama tu contenedor. **Si un archivo trae tus decisiones, es el que se generó a partir de lo que escribiste.** Es la forma de reconocerlo en un servidor que no conoces.

SI EL `PS` NO DEVUELVE NADA

El proceso `java` no está corriendo, o sea que el broker está caído. Míralo con `docker ps --filter name=kafka-broker` y con `docker logs kafka-broker-1 | tail -20`.

Etapa 3 · preguntarle al broker qué está usando

Qué vamos a hacer

El archivo dice lo que se declaró. Ahora vamos a preguntarle al servidor **qué está usando de verdad, y de dónde salió cada valor**.

CONCEPTO · CONFIGURACIÓN EFECTIVA

El valor que el broker **tiene cargado en memoria en este momento**, con la anotación de **de dónde vino**. Un broker tiene cientos de parámetros y tú declaraste unos pocos. Todos los demás están ahí, con su valor de fábrica, decidiendo cosas.

El comando

```
kafka-cli/describe-broker-config.sh 1
```

kafka-cli/describe-broker-config.sh — envoltorio del curso. Por dentro llama a `kafka-configs --describe --entity-type brokers --all`

1 — de qué broker. **Cada uno tiene la suya**, y no tienen por qué coincidir

EL COMANDO REAL, EL QUE VAS A ESCRIBIR EN SUNAT

```
kafka-configs --bootstrap-server kafka-broker-1:29092 \  
--describe --entity-type brokers --entity-name 1 --all
```

--entity-type brokers — preguntamos por un broker, no por un tópico

--entity-name 1 — por cuál

--all — **trae todos los parámetros, no solo los que alguien cambió**. Sin esta bandera la salida sale casi vacía y da la impresión falsa de que el broker no tiene configuración

Los tres orígenes que hay que saber leer

Origen	Significa	Para cambiarlo
DEFAULT_CONFIG	Nadie lo tocó. Es el valor de fábrica	Hay que declararlo
STATIC_BROKER_CONFIG	Vino del archivo, o sea de tus KAFKA_*	Exige reiniciar el broker
DYNAMIC_BROKER_CONFIG	Se cambió en caliente, por la API	Se cambia otra vez en caliente

TRES PROPIEDADES DE LA SALIDA, PARA COMPARAR

```
log.retention.hours=168 sensitive=false
  synonyms={DEFAULT_CONFIG:log.retention.hours=168}

min.insync.replicas=2 sensitive=false
  synonyms={DYNAMIC_DEFAULT_BROKER_CONFIG:min.insync.replicas=2,
            STATIC_BROKER_CONFIG:min.insync.replicas=2,
            DEFAULT_CONFIG:min.insync.replicas=1}

num.partitions=1 sensitive=false
  synonyms={DEFAULT_CONFIG:num.partitions=1}
```

CÓMO SE LEE LA LISTA DE SYNONYMS, Y ES LA CLAVE DEL LABORATORIO

De esa lista manda el primero. Los de más abajo están tapados.

Mira `min.insync.replicas`. El valor efectivo es **2**, y abajo del todo se ve que **el de fábrica era 1**. Tu declaración lo tapó.

Y compáralo con `log.retention.hours`, que trae **un solo origen**. Eso significa que nadie lo tocó nunca, ni tú ni nadie. Es de fábrica y punto.

Cuenta los orígenes de toda la salida

```
docker exec kafka-broker-1 kafka-configs \
  --bootstrap-server kafka-broker-1:29092 \
  --describe --entity-type brokers --entity-name 1 --all \
  | grep -c DEFAULT_CONFIG
```

Repítelo cambiando `DEFAULT_CONFIG` por `STATIC_BROKER_CONFIG` y por `DYNAMIC_BROKER_CONFIG`.

LO QUE SALIÓ EN LA CORRIDA MEDIDA

```
DEFAULT_CONFIG          263
STATIC_BROKER_CONFIG    16
DYNAMIC_BROKER_CONFIG   0

líneas totales          321
```

SI LA SALIDA SALE CASI VACÍA, CON TRES O CUATRO LÍNEAS

Se te perdió el `--all`. **Sin esa bandera Kafka solo lista lo que alguien cambió**, y en un broker recién levantado eso es casi nada. La impresión que da es que el broker no tiene configuración, y es exactamente la lectura equivocada que este laboratorio existe para corregir.

SI LOS `DYNAMIC` NO TE SALEN EN CERO

Alguien dejó un cambio en caliente puesto, y lo más probable es que hayas sido tú en una corrida anterior. Míralo:

```
kafka-cli/describe-broker-config.sh 1 | grep DYNAMIC
```

No es un problema para seguir, pero conviene limpiarlo con `kafka-cli/alter-broker-config.sh 1 <propiedad> --delete` para que el paso 6 se vea limpio. **Y fíjate en lo que acaba de pasar**, porque es el laboratorio entero en una frase. Encontraste un cambio que nadie había escrito en ninguna parte.

VAS BIEN SI...

Los `DYNAMIC` están en cero, porque todavía nadie cambió nada en caliente. Ese cero es tu punto de partida.

Y mira la proporción. **Declaraste dieciséis cosas y el broker está decidiendo con doscientas sesenta y tres que no miraste nunca**. A ti los números te pueden salir algo distintos.

El drift · la propiedad que el archivo no conoce

Qué vamos a hacer

Aquí empieza el momento del laboratorio. Vamos a elegir una propiedad que **tú nunca declaraste**, comprobar que el archivo no sabe nada de ella, y cambiarla en caliente.

La propiedad es `num.replica.fetchers`, que es **cuántos hilos usa el broker para copiar réplicas de otros brokers**. Es la que se sube cuando la replicación va lenta, y la vas a volver a ver en el laboratorio 08.

Pregúntale al archivo si la conoce

```
docker exec kafka-broker-1 sh -c \  
"grep -c num.replica.fetchers /etc/kafka/kafka.properties"
```

grep -c — la `-c` cuenta cuántas líneas coinciden, en vez de mostrarlas

LO QUE SALE

```
0
```

Cero. Nunca la escribiste, así que no está en la etapa 2.

Y ahora pregúntale al broker

```
kafka-cli/describe-broker-config.sh 1 | grep num.replica.fetchers
```

LO QUE SALE

```
num.replica.fetchers=1 sensitive=false  
synonyms={DEFAULT_CONFIG:num.replica.fetchers=1}
```

VAS BIEN SI...

El archivo dice que no la conoce y el broker dice que vale 1. Las dos cosas son ciertas a la vez.

Su valor viene de fábrica, y por eso el único origen que aparece es `DEFAULT_CONFIG`.

El broker te dice que no

Antes de ejecutar, apuesta

Vas a subirla de 1 a 4. **Escribe tu apuesta antes de teclear.** ¿Kafka acepta cualquier número, o hay un límite?

El comando

```
kafka-cli/alter-broker-config.sh 1 num.replica.fetchers 4
```

`kafka-cli/alter-broker-config.sh` — envoltorio del curso. Por dentro llama a `kafka-configs --alter`. **Este envoltorio no te protege del error a propósito**, porque el error es la lección

`1` — sobre qué broker

`num.replica.fetchers 4` — la propiedad y su valor nuevo

LO QUE SALE · UNA PARED DE TEXTO JAVA, Y AQUÍ ESTÁN LAS DOS LÍNEAS QUE IMPORTAN

```
org.apache.kafka.common.errors.InvalidRequestException: Invalid value
org.apache.kafka.common.config.ConfigException: Dynamic thread count update
validation failed for num.replica.fetchers=4, value should not be greater
than double the current value 1 for configuration Invalid dynamic configuration
```

CÓMO SE LEE UN ERROR DE JAVA, Y SIRVE PARA TODO EL CURSO

Las dos primeras líneas, y sáltate la traza. Todo lo que empieza por `at org.apache...` es el recorrido interno del programa y no te dice nada útil.

Aquí el motivo está completo en la segunda línea. **El valor no puede ser más del doble del actual**, y el actual es 1.

Y ESTO NO ES UN TROPIEZO TUYO · ES LA LECCIÓN DEL PASO

Los cambios dinámicos tienen su propia validación, y hay un broker vivo del otro lado diciéndote *eso no, así no*.

Un `server.properties` en disco **no te contesta**. Puedes escribir cualquier disparate y no te enteras hasta el reinicio, que es cuando el broker no arranca. **Un broker corriendo te lo dice en el momento**.

Ahora el mismo cambio, dentro del límite

El doble de 1 es 2:

```
kafka-cli/alter-broker-config.sh 1 num.replica.fetchers 2
```

LO QUE SALE

```
Completed updating config for broker 1.
```

VAS BIEN SI...

El primer comando **falló con el motivo escrito en la segunda línea**, y el segundo dijo

```
Completed updating config for broker 1.
```

Compara con tu apuesta. Si dijiste que aceptaba cualquier número, acabas de aprender algo que no estaba en ninguna documentación que hayas leído.

SI EL SEGUNDO COMANDO TAMBIÉN FALLA

Comprueba el valor actual con `kafka-cli/describe-broker-config.sh 1 | grep num.replica.fetchers`. **El límite es el doble de lo que valga ahora**, así que si en algún intento anterior quedó en otro número, el techo se movió.

El drift, en pantalla

Qué vamos a hacer

El cambio entró. Ahora hay que mirar **las dos etapas por separado**, y ahí está todo el laboratorio.

Primero, la etapa 3 · lo que el broker usa

```
kafka-cli/describe-broker-config.sh 1 | grep num.replica.fetchers
```

LO QUE SALE

```
num.replica.fetchers=2 sensitive=false
synonyms={DYNAMIC_BROKER_CONFIG:num.replica.fetchers=2,
          DEFAULT_CONFIG:num.replica.fetchers=1}
```

Aparecieron los dos orígenes. El `DEFAULT_CONFIG` de fábrica sigue ahí abajo, con su 1, y el `DYNAMIC_BROKER_CONFIG` le gana encima con el 2. **El primero de la lista es el que manda.**

Ahora la etapa 2 · el archivo

```
docker exec kafka-broker-1 sh -c \
  "grep -c num.replica.fetchers /etc/kafka/kafka.properties"
```

LO QUE SALE

```
0
```

AHÍ ESTÁ EL DRIFT

La etapa 3 se movió y la etapa 2 ni se enteró.

EN EL RESTAURANTE

Le gritaste a la cocina «**esa salsa, sin sal**». El cocinero te hizo caso y el plato salió sin sal. **Y la pizarra del menú sigue diciendo lo mismo que decía.** Mañana entra otro cocinero, lee la pizarra, y la sal vuelve.

QUÉ SIGNIFICA ESTO PARA SUNAT

Un cambio dinámico **sobrevive a lo que el broker haga en caliente y no sobrevive necesariamente a todo**. Vive en los metadatos del clúster, no en tu compose ni en tu repositorio de configuración.

Y de ahí sale la regla operativa. **Todo cambio en caliente se escribe además en el archivo, en el mismo turno**, aunque no haga falta reiniciar para que tome efecto. Si no, el que reconstruya ese servidor desde el repositorio va a levantar otro servidor distinto, y nadie va a saber por qué.

LO LOGRASTE SI...

Puedes señalar dos salidas que se contradicen y explicar por qué las dos son correctas.

El archivo dice lo que se declaró. El broker dice lo que está usando. **Solo la segunda es la verdad operativa.**

Dejarlo como estaba

El comando

```
kafka-cli/alter-broker-config.sh 1 num.replica.fetchers --delete
```

--delete — quita el cambio dinámico. **No pone la propiedad en cero**, la devuelve al valor que tenía antes de que la tocaras

Comprueba que volvió

```
kafka-cli/describe-broker-config.sh 1 | grep num.replica.fetchers
```

LO QUE SALE

```
num.replica.fetchers=1 sensitive=false  
synonyms={DEFAULT_CONFIG:num.replica.fetchers=1}
```

LO LOGRASTE SI...

El `DYNAMIC_BROKER_CONFIG` **desapareció de la lista** y volvió a quedar solo el `DEFAULT_CONFIG`, con su 1.

Ese es el camino que hay que saber reconocer. Un `--delete` **devuelve el valor al de la capa de abajo**, que casi nunca es el que alguien quería.

Lo que aprendiste

1 El archivo que manda no siempre se llama `server.properties`.

En la imagen de Confluent el broker arranca con `kafka.properties`, que genera la imagen al encenderse traduciendo tus variables `KAFKA_*`. En un servidor RHEL con el paquete instalado, sí es `server.properties`. **Se comprueba mirando la línea de arranque del proceso, no adivinando.**

2 De la lista de `synonyms` manda el primero.

Los de más abajo están tapados, pero siguen ahí y reaparecen cuando quitas el de arriba. Leer esa lista de arriba abajo es leer la historia de quién tocó ese parámetro.

3 Un broker vivo valida y te contesta. Un archivo no.

Los cambios dinámicos tienen su propia validación, y el broker rechaza lo que no puede hacer explicando por qué. En un archivo puedes escribir cualquier cosa, y te enteras en el próximo reinicio.

4 Todo cambio en caliente se escribe además en el archivo.

En el mismo turno, aunque no haga falta reiniciar. Si no, el drift queda instalado y el que reconstruya ese servidor va a levantar otro distinto sin saberlo.

Para profundizar

A · La traducción, en los dos sentidos

Toma **tres** variables `KAFKA_*` del compose del laboratorio 02 y **predice la propiedad resultante antes de verificarla**. Después haz el camino inverso, eligiendo una propiedad del archivo y reconstruyendo qué variable la generó.

```
docker exec kafka-broker-1 sh -c "grep -E '^(tu.propiedad)= ' /etc/kafka/kafka.properties"
```

Lo que hay que mirar. La regla no tiene excepciones, y por eso se puede predecir. Quitar `KAFKA_`, minúsculas, y `_` a punto.

B · Los tres brokers no tienen la misma configuración

```
kafka-cli/describe-broker-config.sh 1 | grep -E "^ (node.id|broker.id)=\""
kafka-cli/describe-broker-config.sh 2 | grep -E "^ (node.id|broker.id)=\""
kafka-cli/describe-broker-config.sh 3 | grep -E "^ (node.id|broker.id)=\""
```

LO QUE SALE · RECORTADO A LO ANCHO

```
broker 1    node.id=1    synonyms={STATIC_BROKER_CONFIG:node.id=1}
broker 2    node.id=2    synonyms={STATIC_BROKER_CONFIG:node.id=2}
broker 3    node.id=3    synonyms={STATIC_BROKER_CONFIG:node.id=3}
```

Lo que hay que mirar. Tres valores distintos, y los tres con origen `STATIC_BROKER_CONFIG`, porque salieron del compose. **La configuración es por broker.** Un cambio dinámico en el 1 no toca al 2 ni al 3. **Es la causa de los clústeres que se comportan distinto según a qué broker le pegue el cliente**, y de los incidentes más difíciles de reproducir que vas a ver.

C · Un cambio dinámico para todo el clúster

Existe una tercera forma, que aplica a todos los brokers a la vez:

```
docker exec kafka-broker-1 kafka-configs \
  --bootstrap-server kafka-broker-1:29092 \
  --alter --entity-type brokers --entity-default \
  --add-config num.replica.fetchers=2
```

--entity-default — en vez de **--entity-name 1**. Se aplica a todos los brokers que no tengan un valor propio

LO QUE SALE

```
Completed updating default config for brokers in the cluster.
```

Y al describirlo:

```
num.replica.fetchers=2 sensitive=false
synonyms={DYNAMIC_DEFAULT_BROKER_CONFIG:num.replica.fetchers=2,
          DEFAULT_CONFIG:num.replica.fetchers=1}
```

Lo que hay que mirar. Aparece como `DYNAMIC_DEFAULT_BROKER_CONFIG`, que es un **cuarto origen**, distinto del `DYNAMIC_BROKER_CONFIG` del recorrido. **Ya lo habías visto sin buscarlo**, en el paso 3, en la línea de `min.insync.replicas`.

Para deshacerlo, el mismo comando con `--delete-config`:

```
docker exec kafka-broker-1 kafka-configs \
  --bootstrap-server kafka-broker-1:29092 \
  --alter --entity-type brokers --entity-default \
  --delete-config num.replica.fetchers
```

D · Qué es *read-only* y qué no

No todos los parámetros se pueden cambiar en caliente. Pruébalo con uno que no:

```
kafka-cli/alter-broker-config.sh 1 log.dirs /otra/ruta
```

LO QUE SALE

```
org.apache.kafka.common.errors.InvalidRequestException:
Cannot update these configs dynamically: Set(log.dirs)
```

Lo que hay que mirar. El mensaje es **distinto del que viste en el paso 5**. Aquel decía que el valor no era válido. Este dice que **esa propiedad no se puede cambiar en caliente, punto**, sea cual sea el valor.

`log.dirs` es *read-only*. **Cambiar dónde viven los datos con el broker corriendo no tiene sentido**, y Kafka prefiere no dejarte. Los *read-only* exigen editar el archivo y reiniciar, que es justamente lo que en un clúster se planifica.

E · El reporte del laboratorio

`plantillas/reporte-entregable.md`, con las respuestas de referencia en `soluciones/reporte-resuelto.md`.

Antes de cerrar

DEJA EL TERRENO LIMPIO

Comprueba primero que no dejaste ningún cambio dinámico puesto, que es exactamente la higiene que este laboratorio enseña:

```
kafka-cli/describe-broker-config.sh 1 | grep DYNAMIC
```

Si devuelve algo que tú pusiste, quítalo con `--delete`. Y después:

```
bin/stop-mi-cluster.sh
```

LO QUE TE LLEVAS

La próxima vez que alguien diga «pero si en la configuración dice X», ya sabes cuál es la pregunta.
¿En cuál de las tres, y se lo preguntaste al broker o lo leíste en un archivo?